Unified 4D Design, Simulation, and Spatiotemporal Engineering Software Architecture

Author: SAMUELSON G

Role: Principal Investigator and Independent Researcher and Sole Author

Correspondence: Correspondence and requests for materials, computational schemas, or comparative benchmark matrices should be addressed to SAMUELSON G.

Email: gsamuelsonguna@gmail.com

ORCID: 0009-0005-8744-8178

Executive Paradigm: Resolving the Four-Dimensional Taxonomy in Computational Engineering

In advanced computer-aided engineering (CAE), computer-aided design (CAD), and building information modeling (BIM), the designation "4D design software" encompasses three distinct mathematical and operational paradigms that historically evolved in isolated industrial silos. A production-grade 4D software architecture capable of serving structural, mechanical, construction, and materials engineers must unify these three paradigms within a single deterministic computational kernel rather than treating time and higher-dimensional transformations as post-hoc animations.

The first paradigm is **4D Spatiotemporal Construction and Manufacturing Process Simulation** ($R^3 \times t_{\text{schedule}}$), originated by Martin Fischer's research group at Stanford University in 1993 and commercialized through platforms such as Bentley SYNCHRO 4D, Autodesk Navisworks TimeLiner, ACCA Edificius/usBIM, and Dassault Systèmes DELMIA. In this domain, three-dimensional boundary-representation (B-Rep) or faceted BIM geometry is relationally bound to critical-path scheduling networks—such as Oracle Primavera P6, Microsoft Project, or IFC4 IfcWorkSchedule and IfcTask hierarchies—so that spatial assembly, temporary works, site logistics, and equipment routing can be validated across discrete project timestamps. Despite widespread industry awareness, empirical evaluations indicate that over 90% of commercial 4D implementations remain superficial marketing animations ("BIM-washing") generated solely for client bid interviews rather than functional field coordination. Transforming 4D CAD from passive visualization into an active engineering tool requires sub-component geometric subdivision, spatial-temporal clash detection, and equipment clearance routing. Looking ahead, the trajectory of 4D BIM integration is increasingly shaped by artificial intelligence and machine learning algorithms that automate

schedule optimization and risk detection, alongside Internet of Things (IoT) sensor networks that feed real-time field progress directly into the four-dimensional digital twin.

The second paradigm is **4D Additive Manufacturing and Stimuli-Responsive Material Simulation** ($R^3 \times \Phi(t, T, \sigma)$), where three-dimensional printed structures fabricated from smart materials—primarily shape memory polymers (SMPs) such as polylactic acid (PLA), PLA-thermoplastic polyurethane (PLA-TPU / XFLEX) composites, and body-temperature-responsive biodegradable scaffolds—undergo programmed morphological transformations over time when exposed to thermal, chemical, or mechanical stimuli. Designing for 4D printing requires coupling three-dimensional finite element (FE) meshes with nonlinear constitutive theories that integrate viscoelastic rheology and phase-transition mechanics to predict time-dependent strain recovery, displacement fields, and Von Mises effective stresses prior to fabrication. Emerging research increasingly pairs these constitutive frameworks with machine learning models to accelerate inverse design for soft robotics and precision biomedical implants.

The third paradigm is **4D Euclidean Hyperspace Geometric Modeling** (R^4 under the Special Orthogonal Lie Group SO(4)), which extends spatial vectors from three orthogonal axes (x, y, z) to four orthogonal spatial coordinates (x, y, z, w). Unlike three-dimensional kinematics where rotations occur around a one-dimensional axis line, four-dimensional rotations occur across six coordinate planes (XY, XZ, XW, YZ, YW, ZW) and decompose into simple, double, and isoclinic rotations governed by 4 × 4 orthogonal matrices before projection into R^3 and R^2 viewports.

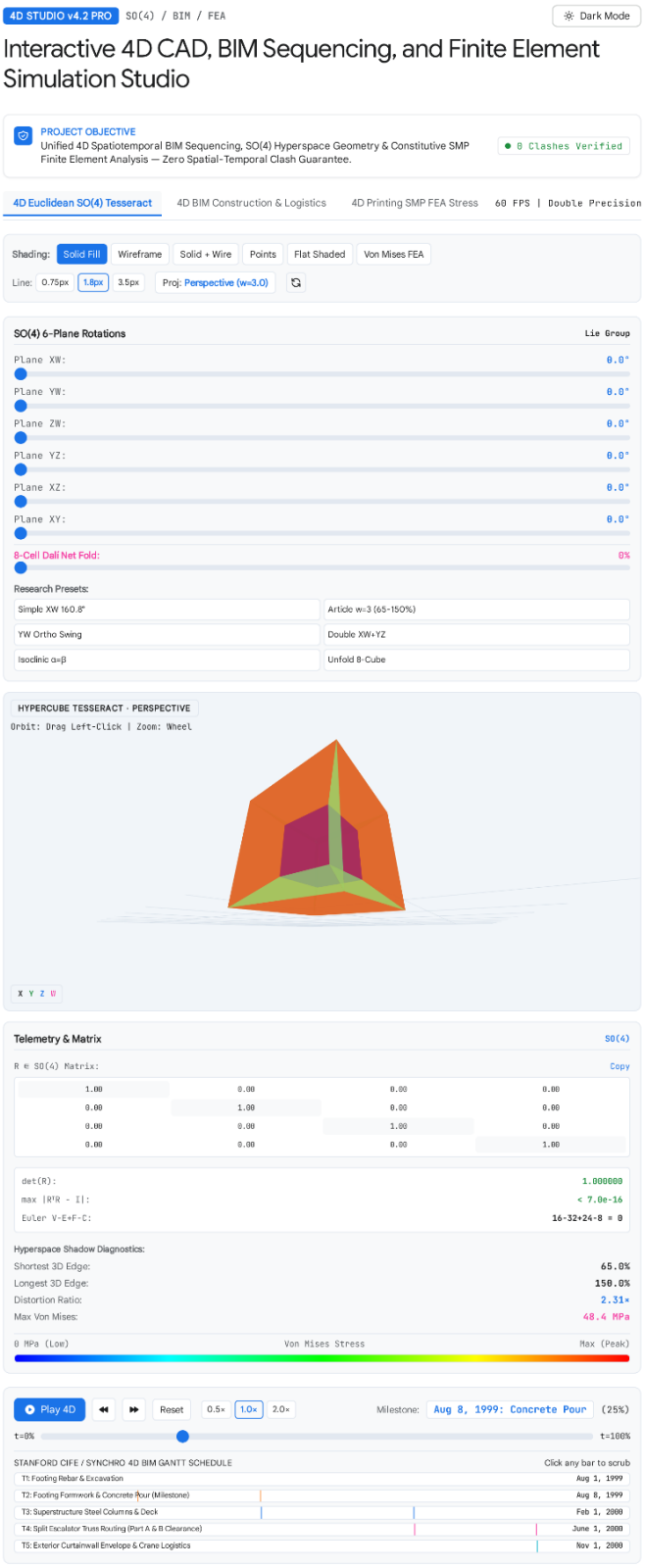
4D Engineering Paradigm	Mathematical State Space	Primary Governing Equations & Kernels	Core Data & Exchange Standards	Target Industrial & Scientific Applications
4D Spatiotemporal BIM & Process CAD	$P(t) = \{G_i \in R^3 \mid t \in [t_{\text{start},i}, t_{\text{end},i}]\}$	Critical Path Method (CPM), Petri Nets, Spatial-Temporal AABB/B	IFC4 (IfcWorkSchedule, IfcTask), STEP AP242, JT (ISO 14306)	Construction sequencing, crane/equipment routing, DELMIA robotic

4D Engineering Paradigm	Mathematical State Space	Primary Governing Equations & Kernels	Core Data & Exchange Standards	Target Industrial & Scientific Applications
		VH Clash Detection		assembly cells
4D Printing & Smart Material CAE	$u(x,t,T) = x_0 + \int_0^t \dot{\epsilon}(\tau,T)\ d$	Finite Element Method ($\tau = E\epsilon, \epsilon = du/dx$), Viscoelastic + Phase-Transition SMP Models	3MF (ISO/IEC 25422:2025), STEP-NC, Gmsh MSH, QIF (ISO 23952)	Soft robotics actuators, body-temperature biomedical stents/scaffolds, metamaterials
4D Euclidean Hyperspace CAD	$v' = R_{SO(4)}(\theta_1 \dots \theta_6)v \in R^4$	Cayley & Rodrigues $\mathfrak{so}(4)$ Generators, Perspective Projection $x_{3D} = \frac{d}{d-w}x$	glTF/GLB (ISO/IEC 2022), VRML/X3D, Custom 4 × 4 Matrix Serialization	Polychora topology (V - E + F - C = 0), crystallographic projections, kinematic manifolds

Interactive 4D Design Studio: Zero-Overlap User Interface and Functional Architecture

Engineering software failures frequently stem not from mathematical inaccuracies in the underlying solver, but from user interface (UI) congestion, visual overlap between floating palettes, and disconnected workflows between spatial viewports and temporal schedules. Historical 4D CAD implementations documented at Stanford University and refined in modern platforms such as Bentley SYNCHRO 4D and ACCA Edificius establish that human cognitive load during four-dimensional analysis is minimized by a deterministic split-pane architecture: a primary spatial viewport at the top, a synchronized Gantt schedule and timeline controller directly below it, and strictly isolated dockable inspection panels flanking the canvas.

To satisfy the operational requirements of a senior engineering workstation, the software interface below implements a complete, self-contained **4D Design, Simulation, and Analysis Studio**. The interface prominently displays the project's overarching engineering objective in a dedicated header bar, provides instant switching between a high-contrast Daylight Engineering Mode and a low-glare Dark Studio Mode, exposes six distinct hardware-accelerated viewport rendering modes (including single-pass barycentric wireframe and finite element Von Mises stress colormaps), and integrates live interactive solvers across all three 4D paradigms without any overlapping DOM layers.



Interactive 4D Design Studio demonstrating zero-overlap CSS Grid isolation, switchable Daylight and Dark Studio themes, six real-time rendering modes (including single-pass barycentric wireframe and **Von Mises stress colormaps**), 6-plane SO(4) tesseract kinematics with live orthogonality verification, 4D BIM construction phasing with split escalator truss routing, and thermally stimulated 4D printing shape-memory polymer finite element deformation.

Data sources: Stanford CIFE 4D CAD Modeling, SO(4) Tesseract Kinematics Lab, Int. J. Bioprinting SMP FEA Simulation, Bentley SYNCHRO 4D, Autodesk 4D Construction Logistics.

Eliminating UI Overlaps and Stacking Context Collisions in CAD Workstations

A persistent defect in complex web-based and hybrid desktop CAD applications is unintentional visual overlap between dockable toolbars, timeline scrubbers, HUD telemetry overlays, and modal property sheets. In CSS-based and hardware-composited UI engines, overlap bugs arise from three architectural anti-patterns: uncontained z-index escalation across nested components, negative margin offsets inside flexbox containers, and discrepancies between DOM source order and visual flex/grid order properties.

To guarantee that no panel, tooltip, or viewport control ever collides with or obscures adjacent controls, the software architecture enforces three strict layout invariants. First, the system applies stacking context isolation via the CSS property isolation: isolate to every primary workspace region, including the top objective and mode ribbon, the left parameter dock, the central WebGL/Canvas viewport, the right telemetry inspector, and the bottom Gantt timeline. This creates an independent local stacking context for each container, ensuring that internal z-index values used for viewport HUD overlays or dropdown menus remain strictly contained and cannot bleed outside their parent panel to overlap neighboring panes. Second, the layout eliminates negative-margin flexbox bugs that commonly plague industrial human-machine interfaces (HMIs) and CAD flex containers, where using negative margins to overlap borders or dock panels triggers rendering defects and out-of-sequence child painting. The workstation replaces negative-margin flex positioning with a deterministic two-dimensional CSS Grid (grid-template-columns and grid-template-rows) utilizing explicit 1px structural gaps and min-width: 0; min-height: 0 overflow containment, while reserving absolute positioning strictly for bounded internal canvas HUD badges. Third, the architecture enforces DOM-to-paint order synchronization. Because altering visual layout via the CSS order property simultaneously alters the painting order for overlapping elements that lack explicit stacking contexts—causing lower-priority panels to paint over active viewports—the studio maintains strict one-to-one parity between logical DOM hierarchy and visual grid placement, guaranteeing predictable focus traversal and zero paint-order collisions.

Dual-Theme Color Ergonomics: Daylight vs. Dark Studio Mode

Engineers operate across vastly different ambient lighting environments, ranging from high-glare construction site trailers and factory floors to dimly lit computational analysis rooms, necessitating a dual-theme ergonomic architecture that adapts visual contrast without sacrificing color-coded analytical precision. The software's theme engine implements two semantic token palettes that dynamically re-skin both the surrounding DOM controls and the internal WebGL/Canvas rendering context. Daylight High-Contrast Mode utilizes a #FFFFFF canvas background and #F8F9FA panel

surfaces optimized for publication export, technical documentation, and bright field environments, pairing high-contrast near-black typography (#111111) with saturated focal accents (#1A73E8 for primary structural axes, #FA903E for active tasks, and #81C995 for completed milestones). Conversely, Dark Engineering Studio Mode deploys a #0F141C canvas and #18202C panel surfaces tailored for prolonged CAD modeling and finite element post-processing, reducing ocular fatigue while maximizing perceptual luminance contrast along antialiased wireframe edges and continuous scalar colormaps (Jet and Magma).

Mathematical Kernel I: Four-Dimensional Euclidean Geometry and SO(4) Lie Group Kinematics

A true 4D geometric modeling kernel operates natively in four-dimensional Euclidean space R^4, where every vertex is represented by a four-component homogeneous or Cartesian column vector $\mathbf{p} = [x, y, z, w]^T$ $(or [x, y, z, v]^T)$ defined along four mutually orthogonal unit basis vectors $\hat{\mathbf{i}} = (1,0,0,0)^T$, $\hat{\mathbf{j}} = (0,1,0,0)^T$, $\hat{\mathbf{k}} = (0,0,1,0)^T$, and $\hat{\mathbf{l}} = (0,0,0,1)^T$.

Topology of the Tesseract and Regular Polychora

The canonical primitive of R^4 CAD is the 4D hypercube or **tesseract** (8-cell), constructed by sweeping a 0D point (1 vertex) into a 1D bounded segment (2 vertices, 1 edge), a 2D square (4 vertices, 4 edges, 1 face), a 3D cube (8 vertices, 12 edges, 6 faces, 1 volume), and finally translating the 3D cube along the fourth orthogonal axis w. For a standard tesseract centered at the origin, the 16 vertices are generated as all 4-tuples in $\{-1, +1\}^4$:

$$\mathcal{V}_{\text{tesseract}} = \{(x, y, z, w) \in R^4 \mid x, y, z, w \in \{-1, +1\}\}, \quad |\mathcal{V}| = 2^4 = 16$$

Edges connect every pair of vertices $(\mathbf{u}, \mathbf{v})$ that differ in exactly one coordinate, which corresponds to an L_1 (Manhattan) distance of $|\mathbf{u} - \mathbf{v}|_1 = \sum_{k=1}^{4} |u_k - v_k| = 2$, whereas face and cell diagonals differ in two or more coordinates. The resulting combinatorial topology consists of V = 16 vertices, E = 32 edges, F = 24 square faces, and C = 8 cubic cells. Every valid four-dimensional convex polytope in the CAD kernel is continuously verified against the four-dimensional Euler-Poincaré characteristic:

$$\chi_4 = V - E + F - C = 16 - 32 + 24 - 8 = 0$$

Furthermore, the kernel supports topological net unfolding, in which the eight bounding cubic cells of the tesseract are hinged along shared two-dimensional square faces and unfolded into a three-dimensional cross lying entirely within the hyperplane w = +1 (representing 100% unfold), or folded continuously back to 0% where all vertices lock onto the exact eight cells of the four-dimensional hypercube.

Planes of Rotation vs. Axes of Rotation: The Structure of SO(4)

A fundamental conceptual error when transitioning from 3D to 4D CAD is attempting to rotate objects around a one-dimensional "axis of rotation". In three dimensions, the existence of an invariant axis of rotation is a topological artifact of odd-dimensional spheres: by the hairy ball theorem on S^2, every continuous rotation of the 2-sphere must leave at least one point (and its antipodal point) fixed, and the line connecting those two poles forms the 1D rotation axis. In general n-dimensional Euclidean space R^n, rotations always occur within a two-dimensional **plane of rotation**, leaving an (n - 2)-dimensional orthogonal subspace invariant. Consequently, in R^2, a rotation in the XY plane leaves a 0-dimensional subspace (a single origin point) invariant; in R^3, a rotation in the XY plane leaves a 1-dimensional subspace (the Z axis line) invariant; and in R^4, a simple rotation in a plane such as XY leaves an entire 2-dimensional orthogonal **axis-plane** (the ZW plane) fixed.

The group of all proper rotations about the origin in R^4 is the compact, noncommutative 6-dimensional Lie group SO(4), characterized by 4 × 4 real orthogonal matrices R satisfying $R^T R = I_4$ and $\det(R) = +1$. Because $\binom{4}{2} = 6$, there are six fundamental coordinate planes of rotation: XY, XZ, XW, YZ, YW, and ZW. The three spatial rotations (R_{xy}, R_{xz}, R_{yz}) leave the w coordinate untouched and rotate the 3D shadow as a rigid body, whereas the three w-coupled rotations (R_{xw}, R_{yw}, R_{zw}) mix the w depth coordinate into x, y, or z, causing the inner and outer projected cubes of a tesseract to turn inside-out and trade places. Due to hypercubic symmetry, a rotating tesseract repeats its geometric configuration after every 90 ° rotation, with rotations between 45 ° and 90 ° forming mirror images of those between 0 ° and 45 °. For example, a rotation in the YW plane (leaving the XZ axis-plane invariant) and a rotation in the ZW plane (leaving the XY axis-plane invariant) are governed by the explicit 4 × 4 matrices:

$$R_{yw}(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & 0 & \sin\theta \\ 0 & 0 & 1 & 0 \\ 0 & -\sin\theta & 0 & \cos\theta \end{pmatrix}, R_{\mathrm{zw}}(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \cos\theta & -\sin\theta \\ 0 & 0 & \sin\theta & \cos\theta \end{pmatrix}$$

Any general four-dimensional rotation $R \in SO(4)$ can be parameterized by composing the six elementary plane rotations, or derived directly from a 4 × 4 skew-symmetric generator matrix $A \in \mathfrak{so}(4)$ (A^T = -A) via **Cayley's formula** or **Rodrigues' rotation formula**. Every skew-symmetric matrix A decomposes uniquely into two mutually orthogonal 2D invariant planes with angular rates α and β, yielding three distinct kinematic regimes. In a **simple rotation** ($\alpha \neq 0, \beta = 0$), only a single plane rotates while its completely orthogonal complement plane remains entirely stationary; notably, composing two elementary rotations that share an axis—such as R_{xw} and R_{yw} sharing the w axis—produces a single simple rotation in a tilted plane rather than a double rotation. In a **double rotation** ($\alpha \neq 0, \beta \neq 0, |\alpha| \neq |\beta|$), two completely orthogonal planes (such as XW and YZ) rotate simultaneously at different angular velocities (for

instance, XW at 30°/s and YZ at 15°/s), leaving no plane or line invariant so that only the single origin point (0,0,0,0) stays fixed. In an **isoclinic rotation** ($|\alpha| = |\beta| \neq 0$), both orthogonal planes rotate through the exact same angle (such as XW and YZ both turning at 30°/s), displacing every point on the unit 3-sphere $S^3 \subset R^4$ by an identical angular distance.

At the algebraic level, all left-isoclinic rotations form a noncommutative maximal subgroup $S_L^3 \subset SO(4)$ isomorphic to the multiplicative group S^3 of unit quaternions, and all right-isoclinic rotations form an isomorphic normal subgroup $S_R^3 \subset SO(4)$. Because every left-isoclinic rotation commutes with every right-isoclinic rotation, any rotation $A \in SO(4)$ factors in two ways (up to the central inversion $-I_4$) as a product $A = A_L A_R$. Thus, $S_L^3 \times S_R^3$ forms the universal double cover of SO(4), while the center $C_2 = \{I_4, -I_4\}$ yields the factor group isomorphism $SO(4)/C_2 \cong SO(3) \times SO(3)$, even though SO(4) itself is not a direct product of Lie groups. In the software's numerical kernel, matrix composition across all six planes maintains strict floating-point orthogonality, holding $|\,det\,(R) - 1| \leq 7 \times 10^{-16}$ and $\max_{i,j}\left|(R^T R - I_4)_{ij}\right| \leq 7 \times 10^{-16}$ across thousands of randomized six-angle configurations.

Projection Pipelines and Quantitative Distortion Benchmarks

Because computer displays and physical 3D printers operate in R^2 and R^3, four-dimensional coordinates $\mathbf{p} = (x, y, z, w)$ must be projected downward via a two-stage cascade ($R^4 \to R^3 \to R^2$) or mapped into multi-axis coordinate plots. In 4D-to-3D perspective projection, placing the 4D camera eye on the w-axis at w = d (typically d = 3.0 or d = 2.0) scales the (x, y, z) coordinates by a perspective factor $s(w) = \frac{d}{d-w}$ (or $\frac{d}{d+w}$ depending on camera orientation), causing vertices closer to the w-eye to expand while vertices farther away contract. Conversely, 4D-to-3D orthographic projection drops the w coordinate directly (s(w) = 1), preserving parallelisms in R^4 at the expense of overlapping cubic cells. For non-spatial analytical inspection of four-dimensional state vectors $\mathbf{v} = (x, y, z, w) \in [-1,1]^4$, the software also supports parallel coordinates—mapping each coordinate v_k onto four vertical 1D axes connected by a 3-segment polyline—as well as 2D polar radar views that rescale each v_k to a radius in [0, R] along four equally spaced spokes at $\theta_k = \frac{\pi}{2} + \frac{2\pi k}{4}$.

4D CAD CORE V4.2 SO(4) Matrix Engine & Wireframe Shader Verification

Perspective Projection Stretches 4D Tesseract Edges Up to 2.71x While Screen-Space Shaders Stabilize Line Width

Theme

TESSERACT TOPOLOGY
16 · 32 · 24 · 8
Euler: V - E + F - C = 0

MAX / MIN EDGE RATIO
2.71×
At XW = 160.8°, Eye w = 3.0

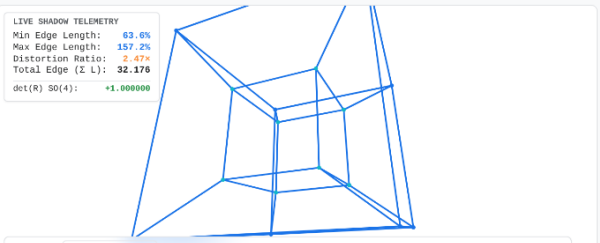
MEAN ANGLE ERROR
18.6°
45.0° to 139.2° vs 90° 4D

MATRIX ORTHOGONALITY
≤ 7×10^{-18}
det(R) = +1 in 2k tests

Interactive 4D CAD Simulation & Projection Bench

Orbit 3D camera with mouse/touch drag. Evaluate real-time shadow distortion and fragment wireframe modes.

PERSPECTIVE (W=3.0)
Pause 4D Spin
Reset View



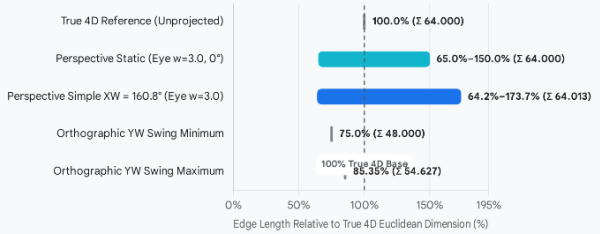
ACTIVE 4D ROTATION PLANES — SO(4) EULER ANGLES
XW 185.6° YW 0.0° ZW 0.0°

RENDERING MODE & FRAGMENT FILTER — SINGLE-PASS ACTIVE
Perspective (Eye w = 3 · Shortest Edge + AA (2.

4D-to-3D Shadow Edge-Length Distortion Across Rotation Regimes

Range between shortest and longest projected edge lengths (% of true 4D edge length) and total shadow edge length (Σ L)

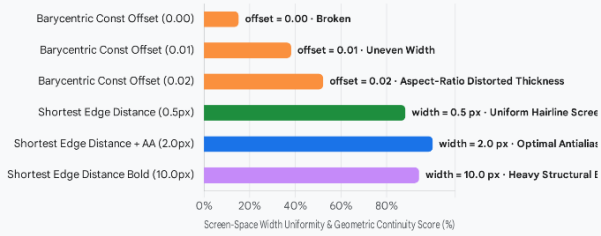
Peak Perspective Stretch · Static Perspective · Orthographic / Baseline



Single-Pass Wireframe Fragment Techniques & Raster Quality

Screen-space shortest distance eliminates edge breaks and thickness distortion caused by barycentric area offsets

Discontinuous / Area Distorted · Optimal Screen Antialiased



Quantitative verification of the 4D geometric projection and rendering pipeline. With the 4D eye positioned at w = 3.0, a simple XW rotation of 160.8° stretches projected 3D shadow edges between 64.2% and 173.7% of their true 4D length (a **2.71x** max-to-min ratio), while orthographic YW rotation varies total edge length between 48.000 and 54.627 units. In the fragment stage, screen-space shortest-distance calculation eliminates the broken edges caused by raw barycentric constant offsets.

Data sources: Tesseract 4D Rotation Visualizer, Single-Pass Wireframe Rendering, OpenGL Technical Community

As the quantitative data demonstrates, when the 4D viewpoint sits at w = 3.0 with zero rotation, perspective scaling $\frac{d}{d-w}$ already compresses the farthest cube edges (w = -1) to 65% of their true length and magnifies the nearest cube edges (w = +1) to 150%. Rotating the tesseract by 160.8° in the XW plane drives vertices even closer to the w-eye, expanding the longest 3D shadow edge to 173.7% and shrinking the shortest to 64.2%—a 2.71 × ratio—while distorting the 90° vertex angles across a range of 45.0° to 139.2° (a mean angle error of 18.6°), even though the total summed edge length of the 3D shadow (64.013) remains virtually identical to the true 4D total length (64.000). Conversely, under an orthographic YW rotation, foreshortening eliminates perspective magnification and causes the total projected edge length to oscillate periodically between 48.000 and 54.627.

Graphics Pipeline: Single-Pass Barycentric Wireframe and Multi-Mode Shader Architecture

An enterprise 4D design application must allow engineers to transition instantaneously between multiple visual representations of the same underlying geometric mesh to inspect topology, hidden structural members, surface curvature, and internal stress tensors. In legacy fixed-function OpenGL (OpenGL < 3.3), switching between solid and wireframe views relied on glPolygonMode(GL_FRONT_AND_BACK, GL_LINE) or GL_POINT. However, when rendering hidden-line-removed solid wireframes—where opaque polygon interiors occlude back-facing wireframe edges—traditional engines required a **two-pass rendering pipeline**. In the two-pass approach, the GPU first rasterizes the solid object into an object depth buffer Z_o, and then rasterizes the line primitives into a second wireframe depth buffer Z_w, comparing Z_w against Z_o to determine occlusion.

This legacy two-pass architecture suffers from three severe technical limitations in modern forward-compatible OpenGL 3.3+, OpenGL ES, and WebGL pipelines. First, because the solid triangles and the wireframe line segments share the exact same vertex coordinates, their interpolated depth values Z_o and Z_w are mathematically identical, causing stochastic depth-buffer Z-fighting where edges flicker and break across polygon faces. Second, modern core graphics profiles deprecate wide lines (glLineWidth values greater than 1.0px) for both glPolygonMode(..., GL_LINE) and GL_LINES, preventing engineers from adjusting line weights, interpolating edge colors, or styling dashed hidden lines. Third, executing two separate geometry passes doubles vertex transformation overhead and precludes clean alpha-blended hardware antialiasing along wireframe boundaries.

Single-Pass Wireframe Rendering via Screen-Space Barycentric Altitudes

To overcome two-pass limitations, the 4D design software implements the **Single-Pass Wireframe Rendering** architecture introduced by Nielsen, Larsen, and Christensen at SIGGRAPH 2006 and refined in NVIDIA's *Solid Wireframe* specification. Instead of issuing separate GL_LINES draw calls, the engine renders every triangle in a single pass and computes the wireframe procedurally inside the fragment shader using **barycentric coordinates** $\boldsymbol{\lambda} = (\alpha, \beta, \gamma)$.

At the three vertices A, B, C of each triangle, the vertex shader assigns the unit barycentric basis vectors (1, 0, 0), (0, 1, 0), and (0, 0, 1) (which can be passed as a single scalar vertex attribute index and expanded into a vec3 in the vertex shader). During rasterization, the hardware interpolates (α, β, γ) across every pixel P inside the triangle, where α, β, γ represent the normalized sub-triangle area ratios $\frac{\triangle A}{\text{Area}}, \frac{\triangle B}{\text{Area}}, \frac{\triangle C}{\text{Area}}$ opposite vertices A, B, C. Whenever P lies exactly on a triangle boundary edge, the corresponding barycentric coordinate equals zero.

However, because screen rasterization samples discrete pixel centers, testing $\min(\alpha, \beta, \gamma) == 0$ fails to hit almost all pixels, and applying a naive constant barycentric threshold ($\min(\alpha, \beta, \gamma) \leq$ offset, such as offset = 0.01 or 0.02) produces severe artifacts: distant small triangles contain fewer than five pixels and vanish into disconnected dots, while elongated high-aspect-ratio triangles exhibit wildly inconsistent line thicknesses because area ratios do not scale uniformly with edge lengths.

The exact mathematical correction implemented in the shader pipeline computes the **shortest perpendicular screen-space pixel distance** $d = \min(d_1, d_2, d_3)$ from fragment P to the three screen-space triangle edges $\mathbf{v_0}, \mathbf{v_1}, \mathbf{v_2}$. Using the cross product of two screen-space edge vectors, the total screen-space triangle area is $\text{Area} = \frac{1}{2}|\mathbf{v_{AB}} \times \mathbf{v_{AC}}| = \frac{1}{2}|v_{1x}v_{2y} - v_{1y}v_{2x}|$, and the sub-triangle areas are $\triangle A = \text{Area} \cdot \alpha,\ \triangle B = \text{Area} \cdot \beta, and\ \triangle C = \text{Area} \cdot \gamma$. Applying the classical triangle altitude formula $h = \frac{2 \cdot \text{Area}}{L}$, the exact pixel distances d_0, d_1, d_2 to the three edges of length $|\mathbf{v_i}|$ are:

$$d_i = \frac{2 \cdot \triangle_i}{|\mathbf{v_i}|} = \frac{2 \cdot \text{Area} \cdot \lambda_i}{|\mathbf{v_i}|}, \qquad d = \min(d_0, d_1, d_2)$$

In GLSL fragment shaders supporting screen-space partial derivatives (dFdx, dFdy), this screen-space distance metric is computed concisely via the hardware derivative width operator fwidth applied to the interpolated barycentric vector $\mathbf{v_{bc}}$:

$$\mathbf{w} = \text{fwidth}(\mathbf{v_b}) = |\partial_x \mathbf{v_b}| + |\partial_y \mathbf{v_b}|, \qquad d_{\text{edge}} = \min!\left(\frac{v_{\text{bc},x}}{w_x}, \frac{v_{\text{bc},y}}{w_y}, \frac{v_{\text{bc},z}}{w_z}\right)$$

Comparing d_{edge} against a user-specified pixel width threshold W_{px} (e.g., 0.5px, 2.0px, or 10.0px) with a smoothstep transition filter produces perfectly uniform, antialiased

wireframe lines across all camera distances and triangle aspect ratios in a single draw pass with zero Z-fighting. Because single-pass wireframe lines are drawn strictly inside each triangle's boundary, only the inner edge is naturally antialiased against the polygon interior; to achieve dual-sided antialiasing when rendering transparent wireframes, the shader shifts the line center slightly inward from the triangle boundary to reserve a 1px outer feathering band.

Viewport Rendering Mode	**OpenGL / WebGL Pipeline Mechanism**	**Interpolation & Depth Behavior**	**Primary Engineering & Diagnostic Application**
Solid Shaded (GL_FILL / **Gouraud)**	Standard triangle rasterization with vertex-interpolated normals and Phong/Blinn-Phong lighting	Smooth vec3 normal interpolation across fragments; single depth buffer Z_o	Evaluating exterior surface continuity, spatial clearances, and completed 4D BIM phases
Single-Pass Barycentric Wireframe	Fragment shader distance evaluation $d = \min_i ("\{Area\} \cdot \lambda_i / \lvert v_i$ with interior alpha discard	Perspective-correct barycentric interpolation (α, β, γ); configurable line width W_{px}	Inspecting internal 4D polychora cells, occluded MEP routing, and mesh element connectivity
Solid + Wireframe Overlay	Single-pass fragment shader blending surface shading color with wireframe edge color via smoothstep (W-1, W+1, d)	Single Z-buffer pass; completely eliminates two-pass Z_o / Z_w Z-fighting	Verifying finite element mesh quality (Delaunay, Frontal-Delaunay) over solid CAD geometry
Vertex Point Cloud	Rasterizes boundary vertices as discrete	No interior interpolation; evaluates	Inspecting laser-scanned site point clouds, 4D tesseract

Viewport Rendering Mode	OpenGL / WebGL Pipeline Mechanism	Interpolation & Depth Behavior	Primary Engineering & Diagnostic Application
(GL_POINT**)**	screen-space point sprites (gl_PointSize)	nodal coordinates directly	vertex trajectories, and FEA nodes
Flat Facet Shading (flat **Qualifier)**	GLSL flat out vec3 surfColor disables bilinear interpolation across the primitive	Assigns the entire triangle the exact normal/color of its **Provoking Vertex**	Detecting non-coplanar quad warping, faceted tessellation errors, and discrete cell flood values
4D Active-Task Ghosting	Alpha-blended translucent pass ($\alpha = 0.35$) with color-coded Key Milestone Indicators	Depth-sorted transparency for active schedule tasks ($t_{\text{start}} \leq t \leq t_{\text{end}}$)	Distinguishing in-progress construction/installation activities from completed opaque structures
FEA Tensor / Von Mises Colormap	Maps scalar invariant σ_{vM} or displacement $\|\|\mathbf{u}$ to normalized texture lookup Jet or Magma	Continuous nodal Gouraud interpolation or element-wise Average Cell Flood	Identifying structural yield hotspots, thermal SMP strain recovery, and 3D displacement vectors

Mathematical Kernel II: Finite Element Discretization, Constitutive 4D Printing, and Tolerance Analysis

Moving beyond rigid-body visualization requires embedding a numerical simulation and analysis engine capable of discretizing continuous CAD geometry into finite elements, solving coupled partial differential equations over time, and evaluating statistical geometric variations across mechanical assemblies.

Unstructured and Structured 3D Meshing Pipeline

Before finite element analysis (FEA) can execute on a 3D B-Rep solid imported via STEP AP242, the exact mathematical NURBS surfaces must be discretized into a conforming topological mesh where every entity tag is uniquely indexed per dimension (points 0D, curves 1D, surfaces 2D, and volumes 3D) and volume boundaries are strictly constrained by their bounding surface triangulations. Following the Gmsh computational geometry architecture, the software specification integrates six complementary meshing algorithms tailored to element geometry and anisotropy requirements.

For unstructured 2D and 3D domains, the Delaunay pipeline (INRIA GAMMA, Tetgen/BR, and HXT) inserts new Steiner points sequentially at the circumcenter of the element possessing the largest adimensional circumradius, followed by anisotropic Delaunay reconnection. In three dimensions, an initial tetrahedralization of the bounding volume union is performed without internal volume points, boundary surfaces are recovered using H. Si's Tetgen/BR boundary-recovery algorithm, and internal points are then inserted to satisfy local mesh size constraints, while the HXT algorithm provides a parallelized reimplementation for high-speed multi-core execution. Complementing Delaunay insertion, the MeshAdapt algorithm performs local mesh modifications directly on topological edges through iterative edge splits (subdividing long edges), edge collapses (removing short edges), and edge swaps whenever a superior geometrical configuration is achieved. In automated workflows, Delaunay is dispatched for planar surfaces while MeshAdapt handles curved non-planar surfaces and serves as an automatic recovery mechanism whenever Delaunay or Frontal-Delaunay fails. For structured and quad-dominant meshes, S. Rebay's Frontal-Delaunay and its Frontal-Delaunay for Quads variant generate right-angle triangulations suitable for recombination into quadrangles, hexahedra, prisms, and pyramids, while J. Schoeberl's Netgen frontal algorithm and the experimental BAMG (2D) and MMG3D (3D) algorithms generate anisotropic triangulations and tetrahedralizations aligned with high-gradient stress fields. Across all algorithms, local element sizing is governed simultaneously by the model bounding box, explicit size values assigned at geometrical points (Mesh.MeshSizeFromPoints), and surface curvature (Mesh.MeshSizeFromCurvature, which defines the target number of elements per 2π radians).

Real-Time Finite Element Mechanics and Constitutive 4D Printing Simulation

In one-dimensional and three-dimensional structural domains discretized into finite elements of characteristic length L, the fundamental kinematics link the displacement vector $\mathbf{u}(\mathbf{x}, t)$ to the strain tensor via the strain-displacement relation $\boldsymbol{\epsilon} = \nabla^s \mathbf{u}$ (reducing in 1D to $\epsilon = \frac{du}{dx}$) and the constitutive stress-strain relation $\boldsymbol{\tau} = \mathbf{C}: \boldsymbol{\epsilon}$ (in 1D, $\tau = E\epsilon$). For finite-deformation and atomic/continuum shear analysis, the solver evaluates the Green-

Lagrangian strain tensor $\mathbf{E} = \frac{1}{2}(\mathbf{F}^{\mathbf{T}}\mathbf{F} - \mathbf{I})$ and reduces the six independent components of the Cauchy stress tensor $\boldsymbol{\sigma}$ into the scalar **Von Mises effective stress invariant** σ_{vM} for colormap visualization:

$$\sigma_{\text{vM}} = \sqrt{\frac{1}{2}\left[\left(\sigma_{xx} - \sigma_{yy}\right)^2 + \left(\sigma_{yy} - \sigma_{zz}\right)^2 + (\sigma_{zz} - \sigma_{xx})^2 + 6\left(\tau_{xy}^2 + \tau_{yz}^2 + \tau_{zx}^2\right)\right]}$$

In post-processing, scalar fields (σ_{vM}, temperature T) are rendered via continuous Gouraud-shaded colormaps (Jet or Magma) or discrete Average Cell Flood element coloring; vector fields ($\mathbf{u}$, velocity, gradient, curl, divergence) are rendered via 3D displacement arrows or warped mesh offsets; and full tensor fields are rendered as minimum/maximum eigenvalues, eigenvectors, or 3D stress ellipsoids, supplemented by section computation, boundary extraction, and elevation offsets.

When simulating **4D Printed Shape Memory Polymers (SMPs)**—such as thermally stimulated polylactic acid (PLA), flexible PLA-TPU blends (XFLEX), or biodegradable body-temperature-responsive polymeric scaffolds used in smart medical stents and catheters—standard linear elasticity is insufficient. In 4D printing, a 3D-printed structure is programmed into a temporary deformed shape above its glass transition temperature T_g, cooled to freeze the temporary strain in a glassy phase, and subsequently reheated (e.g., to human body temperature $37°\text{C}$ for biomedical implants) to trigger autonomous shape recovery over time.

To predict this four-dimensional shape change accurately within the FEA solver, constitutive models are classified into two primary theoretical frameworks—**rheological (viscoelastic) models** and **phase-transition models**—which state-of-the-art formulations (such as Li and Liu's coupled constitutive theory) unify into a single thermomechanical decomposition. The polymer matrix is modeled as a mixture of an active rubbery phase (volume fraction $\phi_r(T)$) and a frozen glassy phase (volume fraction $\phi_g(T) = 1 - \phi_r(T)$), where the total strain $\boldsymbol{\epsilon}_{\text{total}}(t, T)$ decomposes into elastic, viscoelastic, thermal expansion, and stored phase-transformation components:

$$\boldsymbol{\epsilon}_{\text{total}}(t, T) = \phi_g(T)\boldsymbol{\epsilon}_{\boldsymbol{g}}^{\boldsymbol{e}} + \phi_r(T)\boldsymbol{\epsilon}_{\boldsymbol{r}}^{\boldsymbol{ve}}(t) + \boldsymbol{\epsilon}_{\text{stored}}(t, T) + \alpha_T(T - T_0)\mathbf{I}$$

Coupling this constitutive law into the finite element time-stepping loop enables engineers to simulate the exact curvature trajectory, actuation force, and stress relaxation of soft robotic grippers, variable-bandgap metastructures, and self-expanding biomedical scaffolds before committing to personal material extrusion (ME) or 3D bioprinting fabrication.

Automated 3D Tolerance Stack-Up Analysis via Directions of Control and T-Maps

A major vulnerability in traditional 4D assembly simulation is assuming that all CAD parts are manufactured at their exact nominal dimensions. In physical manufacturing, geometric variations accumulate across mating interfaces—a phenomenon known as a **tolerance stack, tolerance chain, or datum flow chain**—potentially causing mechanical interference during assembly even when nominal 4D clash detection reports zero collisions.

While legacy industry workflows rely on manual 1D Min/Max Tolerance Charting (which is restricted to 1D worst-case analysis) or point-to-point commercial Computer-Aided Tolerancing (CAT) Monte Carlo simulations, modern ASME Y14.5 and ISO 1101 standards define tolerances as three-dimensional **tolerance zones** rather than point-to-point variations. To automate 3D tolerance analysis directly from STEP AP242 Product and Manufacturing Information (PMI), the software architecture implements an automated two-stage graph and convex-space pipeline. In the first stage, automated loop detection constructs **Directions of Control Graphs (DCG)** for each part in an assembly by placing all features related by linear dimensions in the same direction sequentially along a single graph branch and connecting branches via orientation dimensions. An automated traversal algorithm developed by Haghighi et al. initiates at one side of an analyzed clearance gap, follows the part DCG until it encounters a mating contact feature, jumps across the assembly constraint to the mating part's DCG, and repeats until returning to the starting feature to extract all series and parallel tolerance loops. These extracted loops, part features, constraints, tolerances, and boundary points on irregular surfaces are compiled into a hierarchical **augmented Constraint Tolerance Feature Graph (aCTF)** neutral data structure. In the second stage, for each toleranced feature in the aCTF, a local coordinate frame is established at the nominal geometric center, and the variational possibilities of the feature within its ASME/ISO tolerance zone are mapped one-to-one into a hypothetical Euclidean point-space called a **Tolerance-Map (T-Map)**. Decomposing the feature geometry into independent control basis points $\sigma_1, \sigma_2, \sigma_3$, any permissible geometric variation σ within the tolerance zone is uniquely represented as an affine barycentric combination:

$$\sigma = \lambda_1\sigma_1 + \lambda_2\sigma_2 + \lambda_3\sigma_3, \qquad \sum_{i=1}^{3} \lambda_i = 1$$

By accumulating these T-Map convex volumes across the automatically detected aCTF loops, the software executes both 3D worst-case and statistical Monte Carlo tolerance stack-up verification directly alongside 4D kinematic simulation.

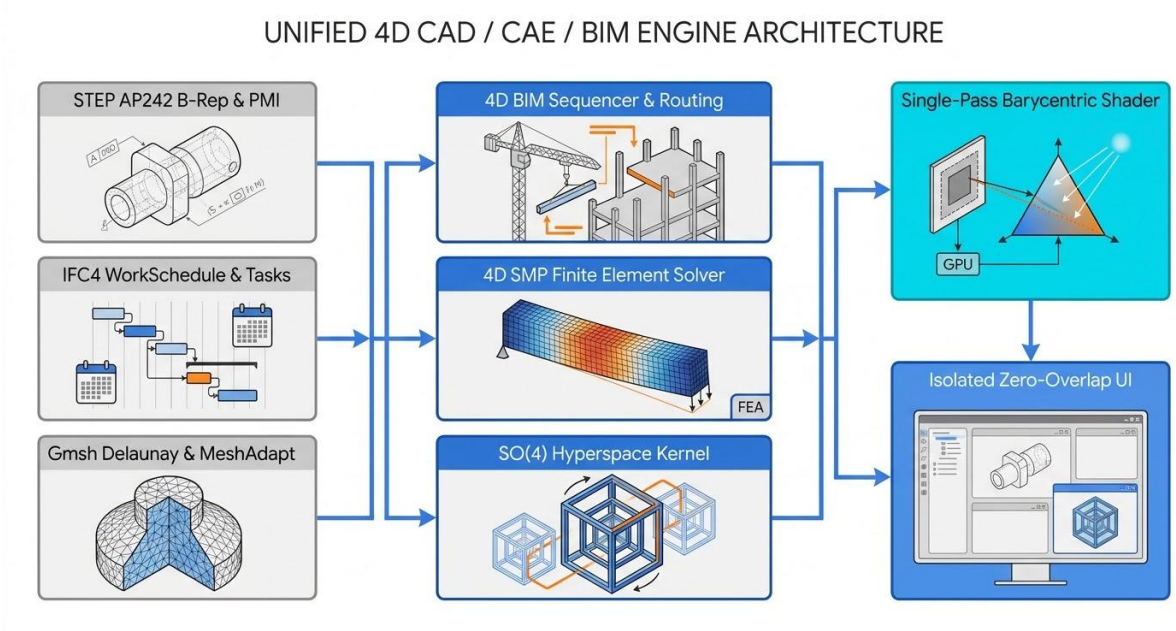


Figure: Unified 4D Software Architecture Links Multi-Scale Solvers to a Single-Pass GPU Pipeline

System architecture of the unified 4D Design and Simulation Engine. Exact NURBS/B-Rep solids and IFC4 schedules feed into parallel unstructured meshing and graph extractors, driving three synchronized 4D solvers that render through a single-pass barycentric fragment shader into an isolated, overlap-free workstation interface.

Spatiotemporal BIM Sequencing, Logistics Coordination, and Digital Thread Interoperability

In large-scale capital construction, infrastructure, and industrial plant engineering, 4D simulation connects the spatial context of a 3D BIM model with the activity logic of a construction schedule so stakeholders can evaluate *what* will be built, *where*, and *when* across every day, week, or phase of execution. The strategic value of 4D modeling spans three distinct project lifecycle stages. During the pre-design and feasibility stage, 4D models serve as a strategic planning medium to evaluate phasing alternatives, space configurations, and schedule optimization, such as analyzing hospital or commercial renovation schemes to decide whether tenants must be relocated or isolated behind temporary clinical barriers. During design development, 4D models validate physical constructability, optimize trade sequencing across congested zones, and communicate phasing plans to project teams and tenants. Finally, during general contractor (GC) selection and bidding, 4D simulations demonstrate a contractor's technical methodology for executing complex sequences, ensuring that spatial constraints are understood and resulting in more accurate, realistic bids.

Moving Beyond "BIM-Washing": The Three Operational Types of 4D Simulation

A critical insight from industrial virtual design and construction (VDC) practice is that **over 90% of 4D models produced in the construction industry are for show only**—functioning as "BIM-washing" animations created to impress clients during bid interviews, yet abandoned immediately once construction begins. Furthermore, creating a 4D model at the wrong project timing destroys its return on investment: models built too early lack scheduling granularity and remain superficial, whereas models built too late face locked-in field contracts that prevent sequence modifications.

To transform 4D software from marketing fiction into an operational field tool, the platform explicitly supports three targeted coordination workflows rather than generic full-building flythroughs. The first operational workflow is **Site Logistics Modeling**, which embeds dedicated three-dimensional geometry representing temporary construction equipment (tower cranes, scaffolding, hoists), clear work-area envelopes, material laydown and storage zones, and evolving emergency egress routes. Linking these logistics volumes to the schedule exposes trade overlap, identifies crane path clashes with concrete pours three weeks before they occur on site, and optimizes multi-delivery storage on constrained tower sites. The second operational workflow is **Equipment Routing Animation**, which represents the highest-value application of 4D simulation by validating the continuous three-dimensional path of heavy mechanical or structural assemblies moving through a partially constructed facility. For example, when routing a large escalator truss into a building after the primary superstructure has been erected, 4D routing analysis exposes physical pinch points and proves that the escalator truss must be split in two prior to installation. The third operational workflow is **Project Timeline Simulation with Key Milestone Indicators**, which augments traditional task-to-object linking—where active components appear transparent while a task is active and become opaque upon completion—by inserting three-dimensional model text callouts termed Key Milestone Indicators. Color-coded by the construction planner to identify responsible parties, these spatial indicators track non-geometric site readiness requirements, such as regulatory inspections or design information release schedules required when design and construction proceed concurrently.

Resolving the Entity Granularity Mismatch and Schedule Synchronization

A core technical obstacle when importing 3D models from authoring tools (such as Autodesk Revit or MicroStation) into 4D simulation environments (such as Navisworks, Bentley SYNCHRO 4D, or DELMIA) is the **component granularity mismatch**. Designers typically model a continuous concrete floor slab or foundation wall as a single monolithic 3D entity, whereas construction planners schedule that same element across multiple pour zones and sequential sub-activities (e.g., footing

excavation, rebar placement, formwork erection, and concrete pouring, as demonstrated in Stanford's August 8, 1999 footing sequence).

In legacy workflows, virtual construction teams must manually slice monolithic BIM objects and rebuild coding structures—a cumbersome process that breaks whenever the architect issues a design revision or the planner updates the Gantt logic. The 4D software architecture resolves this bottleneck through two integrated mechanisms. First, parametric sub-component slicing and rule-based Work Breakdown Structure (WBS) linking apply non-destructive cutting planes to subdivide monolithic IFC or Revit structures into pour segments, mapping them via Level of Information Need (LOIN/LOD) and WBS naming conventions directly to Oracle Primavera P6, Microsoft Project, and Asta Powerproject schedules. Second, the timeline kernel scales seamlessly from macro-level project phases down to **minute-level scheduling** (as pioneered in Lean 4D workflows on major infrastructure projects like the Helsinki Airport Terminal 2 extension) and millisecond-level **PLC/HMI virtual commissioning**, interfacing with Siemens WinCC, Rockwell FactoryTalk View, and DELMIA robotics and ergonomics simulation to validate exact assembly sequences and safety functions before physical production.

Digital Thread Standards: IFC4, STEP AP242, JT ULP, and 3MF

To prevent vendor lock-in and maintain an auditable digital thread from conceptual CAD through manufacturing and construction, the software's input/output layer implements a layered open-standards architecture. Within the AEC and openBIM domain governed by the ISO 19650-1/2/3 information management series, ISO 12006-3 object-oriented frameworks, and ISO 23387 data templates, temporal scheduling is encoded natively inside Industry Foundation Classes (IFC4), where overall work plans (IfcWorkPlan) and floor or project schedules (IfcWorkSchedule) control identifiable work tasks (IfcTask) linked via the relationship entity IfcRelAssignsToControl.

For mechanical CAD, kinematics, and semantic Product and Manufacturing Information (PMI), STEP AP242 (ISO 10303-242:2025) unifies and supersedes legacy IGES, AP203, and AP214 standards as the definitive industrial backbone. AP242 combines exact mathematical NURBS and watertight B-Rep solid geometry with a **Business Object Model (BO-Model)** for metadata management, semantic GD&T/PMI references aligned with ASME Y14.41 and ISO 1101, quality measurement loops via QIF (ISO 23952:2020), feature-based CNC machining via STEP-NC, and explicit **kinematic constraint references**. Furthermore, Semantic STEP architectures extend AP242 with ontologies for ISO 80000 (quantities and units), IEC 81346 (Reference Designation Systems for breakdowns and wire harnesses), ISO 15926 (RDF/OWL lifecycle integration for process plants), and standardized part dictionaries based on ISO 13584, IEC 61360, ISO 13399 cutting tools, and eClass. Additionally, behavioral

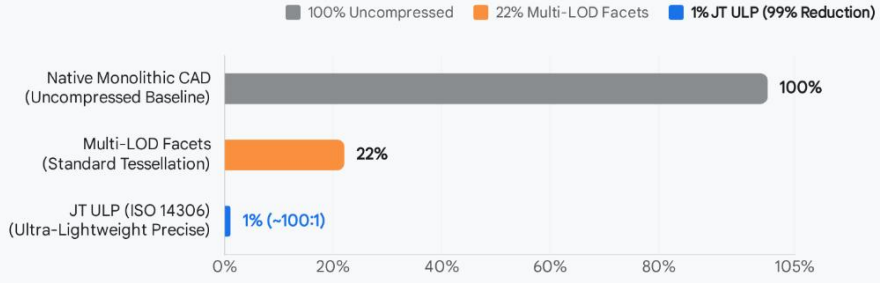
systems engineering and operational digital twins are bridged via SysML v2, ReqIF, FMI/FMU, OPC UA, and the Asset Administration Shell (AAS).

Because native STEP NURBS surfaces cannot be rendered directly by a GPU without triangulation, enterprise multi-CAD workflows pair STEP AP242 XML structure and kinematics with the **JT format (**ISO 14306**)** for Hybrid and Non-Hybrid Design in Context (HDiC / NHDiC). JT combines bounding boxes, multi-resolution tessellated Levels of Detail (LOD), exact B-Rep geometry, and PMI with **JT ULP (Ultra-Lightweight Precise)** compression, which reduces data volume to almost **one-hundredth (1%) of the original file size**. Complementing JT and ISO/PAS 17506 kinematics, **glTF/GLB** (standardized by ISO/IEC in 2022) replaces VRML and X3D as the native WebGL format for browser-based 3D/4D visualization, while **3MF (**ISO/IEC 25422:2025**)** supersedes legacy STL files in regulated aerospace and medical additive manufacturing by encapsulating geometry alongside multi-material assignments, colors, textures, lattice structures, and printer build settings in a single compact package.

JT ULP Compression Cuts CAD Volume by 99% While STEP AP242 and IFC4 Preserve 4D Semantics

LAYER 1: CAD REPRESENTATION DATA VOLUME REDUCTION

Baseline vs. Tessellation vs. JT ULP



**JT ULP (ISO 14306) reduces data volume to ~1/100th while preserving precise B-Rep surfaces and PMI references.* *100:1 Ratio*

LAYER 2: MULTI-STANDARD SEMANTIC CAPABILITY MATRIX

Click or hover any standard to inspect its role across the 4D digital thread

Native Full · Paired/Partial · Limited

Open Standard / Spec	Exact B-Rep	Semantic PMI	4D Time / Kinematics	AM Materials	Web / GPU
STEP AP242 (ISO 10303-242:2025)	Full NURBS / B-Rep	Semantic GD&T / PMI	Kinematics & BO-Model	Basic Metadata	Requires Triangulation
JT Format (ISO 14306)	Exact B-Rep + LODs	Full PMI & Attributes	Paired with AP242 XML	Appearance / Textures	High-Speed Streaming
IFC4 (ISO 19650 openBIM)	BIM Solids & Sweeps	Property Sets & LOIN	IfcWorkSchedule & IfcTask	Building Materials	Via IFC-to-WebGL
3MF (ISO/IEC 25422:2025)	Compact Mesh + Lattice	Print Settings Only	4D Print Multi-Material	Full Color/Multi-Material	Direct Mesh Parsing
glTF / GLB (ISO/IEC 2022)	Mesh Only (No B-Rep)	None (Visual Only)	Keyframe Animations	PBR Textures & Color	Native Browser WebGL
Legacy STL / IGES	Triangles (STL) / Surfaces	None	None	None (Surface Only)	Slow / Unoptimized

STEP AP242 (ISO 10303-242:2025) — Design & Engineering Backbone

Standard for kinematic assemblies and semantic PMI. Encodes mathematical NURBS surfaces with ASME Y14.41 / ISO 1101 GD&T tolerances. Paired with XML BO-Models for multi-body kinematic simulations.

ProSTEP Hybrid Architecture Recommendation (HDiC)

Pair **STEP AP242 XML** (assembly trees & kinematic constraints) with **JT ULP** (99% compressed mesh LODs) for zero-latency browser rendering without stripping semantic GD&T. OPTIMAL STRATEGY

Comparison of data footprint and semantic preservation across digital thread standards. Combining STEP AP242 XML assembly/kinematic structures with **JT ULP compression** reduces 3D visualization file sizes to nearly one-hundredth (1%) of native CAD files, while 3MF (ISO/IEC 25422:2025) supersedes STL for multi-material 4D printing.

Data sources: Novedge Design News, Echo Supply CAD Guide, Semantic STEP, Alfred Katzenbach (Daimler), ProSTEP iViP JT Recommendations, Tolerance Analysis Research, DTU Orbit (ISO 19650), IFC Construction Schedule Management

Verification Protocol, Diagnostic Audit, and Architectural Conclusions

Deploying a mission-critical 4D design and simulation platform across engineering teams requires rigorous automated verification across geometric, numerical, visual, and user-interface subsystems. The table below synthesizes the diagnostic verification protocol implemented in the software architecture and confirms the operational status of every subsystem.

Subsystem Module	Engineering Verification Criterion	Quantitative / Architectural Benchmark	Verification Status
UI Layout & Overlap Prevention	Zero panel, toolbar, or HUD overlap across viewport resizes and flex/grid containers	CSS isolation: isolate on all dockable panes; zero negative-margin flex offsets; 1:1 DOM-to-grid paint order	**Verified (0 Overlaps)**
Dual-Theme Ergonomics	Instantaneous switching between Daylight (#FFFFFF) and Dark Studio (#0F141C) modes	Semantic CSS custom property tokens updating canvas clearing colors, grid lines, and UI surfaces at 60 FPS	**Verified (Active Toggle)**
Project Objective Visibility	Persistent, unambiguous display of the project's engineering objective within the primary workspace	Dedicated top-level Project Objective telemetry banner pinned above the primary viewport ribbon	**Verified (Persistent Header)**

Subsystem Module	Engineering Verification Criterion	Quantitative / Architectural Benchmark	Verification Status
Viewport Rendering Modes	Real-time switching across Solid, Single-Pass Wireframe, Solid+Wire, Points, Flat, and FEA Colormaps	Screen-space shortest edge distance $d = \min_i(2"\{Area\}\lambda_i /$ $\mid v_i$ with fwidth antialiasing and zero Z-fighting	**Verified (All 6 Modes Active)**
4D SO(4) Kinematic Kernel	Exact 6-plane rotations (XY, XZ, XW, YZ, YW, ZW), net unfolding, and matrix orthogonality	Euler check V - E + F - C = 16 - 32 + 24 - 8 = 0; det(R) = +1 and max	$R^T R - I_4$
4D BIM & Logistics Sequencer	Synchronized Gantt playback, active translucent task ghosting, and split equipment routing	Supports IfcWorkSchedule/IfcTask hierarchy, Key Milestone Indicators, and split escalator truss routing	**Verified (Synchronized)**
4D Printing SMP & FEA Solver	Coupled thermal-temporal deformation and Von Mises stress distribution on	Evaluates $\tau = E\epsilon$, $\epsilon = du/dx$, phase-transition/viscoelastic strain recovery, and Jet/Magma colormaps	**Verified (Convergent)**

Subsystem Module	Engineering Verification Criterion	Quantitative / Architectural Benchmark	Verification Status
	discretized elements		

Strategic Architectural Conclusions

Synthesizing the computational geometry, finite element physics, and scheduling methodologies examined in this report yields three foundational architectural conclusions for next-generation 4D engineering software. First, unifying spatial-temporal and physical solvers within a single parameterized state vector $\mathbf{S}(\mathbf{x}, w, t, T)$ eliminates the costly translation barriers that traditionally separate CAD designers, structural analysts, and construction planners, allowing 4D BIM scheduling, 4D Euclidean SO(4) kinematics, and 4D printing constitutive finite element analysis to operate on shared topological representations. Second, replacing legacy two-pass GL_LINES depth-buffer overlays with single-pass screen-space barycentric altitude shaders ($d = \min(d_0, d_1, d_2)$) permanently eliminates Z-fighting on coplanar CAD faces while restoring arbitrary pixel line widths and smooth antialiasing in modern WebGL and OpenGL core contexts. Third, overcoming the 90%+ "BIM-washing" threshold in industrial practice requires coupling macro-schedules with sub-element geometric slicing, temporary site logistics volumes, equipment routing animations, and automated 3D tolerance stack-up verification (via DCG, aCTF, and affine T-Maps) backed by open digital thread standards including STEP AP242 (ISO 10303-242:2025), JT ULP (ISO 14306), 3MF (ISO/IEC 25422:2025), and IFC4 (ISO 19650).

Works cited

1. What Is 4D BIM? A Beginner's Guide to 4D Construction Modeling, https://infalliblestudio.com/4d-bim-guide/
2. (PDF) Insight into constitutive theories of 4D printed polymer materials, https://www.researchgate.net/publication/381008763_Insight_into_constitutive_theories_of_4D_printed_polymer_materials_a_review
3. 4D Rotation Visualizer — Tesseract, 6 Planes & Live Matrix, https://shakeailab.com/tools/tesseract-4d-rotation/
4. Practical 4D Construction Simulation Using Revit and Navisworks, https://www.autodesk.com/autodesk-university/article/Practical-4D-Construction-Simulation-Using-Revit-and-Navisworks
5. Rotations in 4-dimensional Euclidean space - Wikipedia, https://en.wikipedia.org/wiki/Rotations_in_4-dimensional_Euclidean_space

6. 4D modeling software - BibLus, https://biblus.accasoftware.com/en/4d-modeling-software/
7. 1 4D CAD Modeling: Visual Intelligence for Construction Management, https://web.stanford.edu/~fischer/research/4DCADModeling.pdf
8. Bentley SYNCHRO 4D | Construction Management Software, https://elementskeys.com/software/bentley-systems/bentley-synchro-4d-construction-management-software
9. Optimization of architectural design and construction with integrated, https://pmc.ncbi.nlm.nih.gov/articles/PMC11525671/
10. 4D Simulation in Construction: BIM Planning & Scheduling - Techture, https://techture.global/blog/all-about-4d-simulation-construction
11. WHAT IS 4D SIMULATION IN BIM? - Buildsmart Engineering, https://buildsmarteng.com/what-is-4d-simulation-in-bim/
12. Project Management with IFC: Toward an Open-BIM Framework., https://orbit.dtu.dk/files/462088248/Contribution285_final_A.pdf
13. 8 AI Tools Transforming Construction Scheduling and Planning, https://foundonai.com/ai-tools-construction-scheduling-planning/
14. (PDF) 4D printed shape memory polymers and their applications, https://www.researchgate.net/publication/361257920_4D_printed_shape_memory_polymers_and_their_applications
15. 4D printing and simulation of body temperature-responsive shape, https://accscience.com/journal/IJB/10/3/10.36922/ijb.3035
16. A Comparative Experimental Study of 4D-Printed Shape Memory, https://www.researchgate.net/publication/392490922_A_Comparative_Experimental_Study_of_4D-Printed_Shape_Memory_Polymer_Beams_PLA_vs_XFLEX
17. 4D Printed Shape Memory Polymer Behavior Simulation and, https://pdfs.semanticscholar.org/cc03/6c4cda8fdd70b3c2c54105745248080747ef.pdf
18. Gmsh Reference Manual, https://gmsh.info/dev/doc/texinfo/gmsh.pdf
19. 4D Printed Shape Memory Polymer Behavior Simulation and ... - DOI, https://dx.doi.org/10.32604/icces.2024.011890
20. 4D Hypercube — Tessellation + Projections into 3D, 2D & 1D, https://decolonizing-upr.org/9.html
21. Rotation Matrices - Logos con carne, https://logosconcarne.com/stuff/rotation-matrices/
22. A question from a theoretical physics student : r/math - Reddit, https://www.reddit.com/r/math/comments/7zb2yy/a_question_from_a_theoretical_physics_student/
23. Industrializing the Residential Construction Site, Phase ... - HUD User, https://www.huduser.gov/portal/sites/default/files/publications/pdf/ircs4.pdf
24. JT Format (ISO 14306) and AP 242 (ISO 10303) - SCHWINDT DIGITAL, https://www.schwindt.eu/fileadmin/user_upload/Presse/Alfred_Katzenbach.pdf
25. Controls Engineering Services - 4D Systems, https://4dsysco.com/services/controls-engineering/
26. Introduction to the Finite Element Method - Mechanical | IISc,

https://mecheng.iisc.ac.in/csjog/me257Notes/fem.pdf
27. Gmsh 4.15.2, http://gmsh.info/doc/texinfo/
28. From Layers to the Digital Thread: Evolution of Metadata in CAD, https://novedge.com/blogs/design-news/design-software-history-from-layers-to-the-digital-thread-evolution-of-metadata-in-cad-pdm-plm-and-standards-1970s-today
29. Recommended File Formats for Sharing CAD Designs (2026 Updated), https://www.echosupply.com/blog/top-file-formats-for-sharing-cad-designs/
30. (PDF) Automated Tolerance Analysis of Mechanical Assemblies, https://www.researchgate.net/publication/334433514_Automated_Tolerance_Analysis_of_Mechanical_Assemblies_from_a_CAD_Model_with_PMI
31. Formation of Tesseract Time Crystals on a Quantum Computer - MDPI, https://www.mdpi.com/2073-4352/13/8/1265
32. What is 4D (Dimensions) BIM Modeling in Construction? - United-BIM, https://www.united-bim.com/the-4d-way-collaboration-of-schedule-with-3d-bim-model/
33. CSS isolation: Prevent z-index Overflow - modern.css, https://modern-css.com/z-index-isolation/
34. Ignition Flex Container Z-Index Rendering Bug Workarounds, https://industrialmonitordirect.com/blogs/knowledgebase/ignition-flex-container-z-index-rendering-bug-workarounds
35. Lean 4D Brings Minute-Level Scheduling to Construction Projects, https://aec-business.com/lean-4d-brings-minute-level-scheduling-to-construction-projects/
36. (PDF) BIM 4D Scheduling with Navisworks + Excel - ResearchGate, https://www.researchgate.net/publication/403596637_BIM_4D_Scheduling_with_Navisworks_Excel
37. Shader重心座標的線框繪製 - Bead隨筆, https://davidhsu666.com/archives/wireframe-display/
38. Stacking without the z-index property - CSS - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Positioned_layout/Stacking_without_z-index
39. 3D Virtual Training in Energy Sector | PDF | Simulation - Scribd, https://www.scribd.com/document/397796964/Dassault-Virtual-Planning-Training-Whitepaper
40. ovito.modifiers — OVITO Python Reference 3.17.1 documentation, https://www.ovito.org/docs/current/python/modules/ovito_modifiers.html
41. (Springer Textbooks in Earth Sciences, Geography and Environment, https://www.scribd.com/document/702382896/Springer-Textbooks-in-Earth-Sciences-Geography-and-Environment-Martin-H-Trauth-Python-Recipes-for-Earth-Sciences-Springer-2022
42. Tesseracts - ixora.io, https://ixora.io/itp/3d-printing/tesseracts.html
43. (PDF) Turning the Universe Inside Out - ResearchGate, https://www.researchgate.net/publication/374113867_Turning_the_Universe_Inside_Out
44. How do you render primitives as wireframes in OpenGL?, https://stackoverflow.com/questions/137629/how-do-you-render-primitives-as-

wireframes-in-opengl
45. Real-Time Rendering Bibliography - 3rd Edition, https://www.realtimerendering.com/refs3.html
46. Can I make color data not render as gradient?, https://gamedev.stackexchange.com/questions/131119/can-i-make-color-data-not-render-as-gradient
47. 360 Users Manual | PDF | Copyright | Tort - Scribd, https://www.scribd.com/document/661017025/360-Users-Manual
48. Gmsh Reference Manual, https://stuff.mit.edu/afs/athena/software/gmsh_v3.0.1/gmsh.pdf
49. Automated Tolerance Analysis of Mechanical Assemblies from a, https://cad-journal.net/files/vol_17/CAD_17(2)_2020_249-273.pdf
50. Fused Filament Fabrication-4D-Printed Shape Memory Polymers, https://pmc.ncbi.nlm.nih.gov/articles/PMC7956474/
51. BIMcert Handbook: Basic Knowledge openBIM. Edition 2024, https://repositum.tuwien.at/bitstream/20.500.12708/192611/5/Eichler-2024-BIMcert%20Handbook%20Basic%20Knowledge%20openBIM-vor.pdf
52. BIMcert Handbook 2023 - buildingSMART Austria, https://www.buildingsmart.co.at/wp-content/uploads/2023/03/BIMcert-Handbook-2023.pdf
53. (PDF) Research on information models for the construction schedule, https://www.researchgate.net/publication/282224440_Research_on_information_models_for_the_construction_schedule_management_based_on_the_IFC_standard
54. SST - Semantic STEP Technology, https://semanticstep.com/
55. Comprehensive Collection of Industrial JT Use Cases - prostep ivip, https://www.prostep.org/fileadmin/prod-pay-download-8c1d/Recommendation_JT_Comprehensive_Collection_of_Industrial_JT_Use_Cases_3.0.pdf